

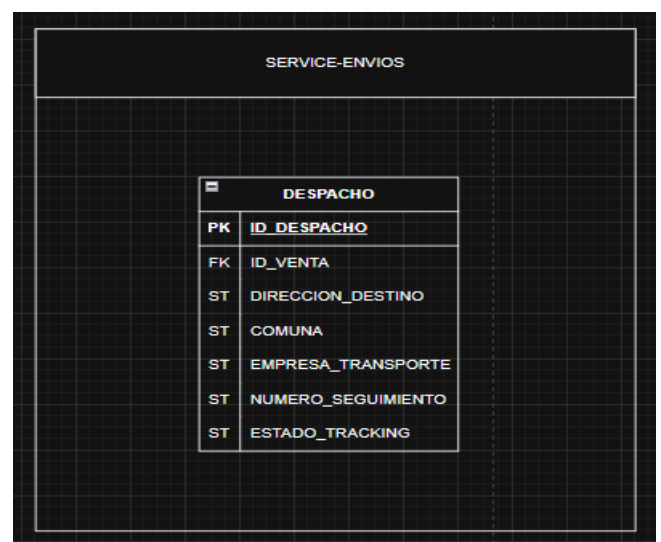
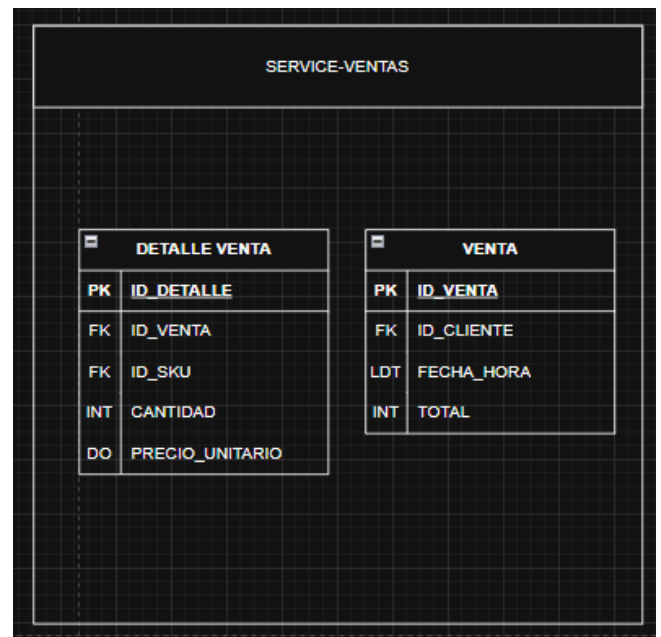
DesireeStore

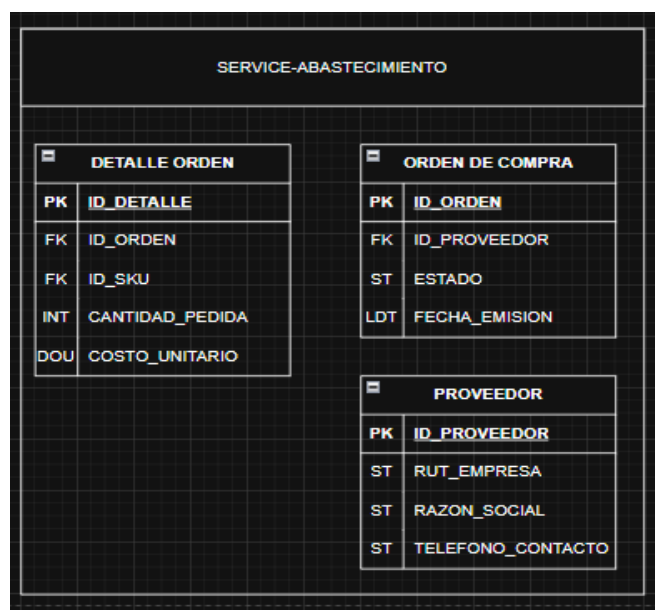
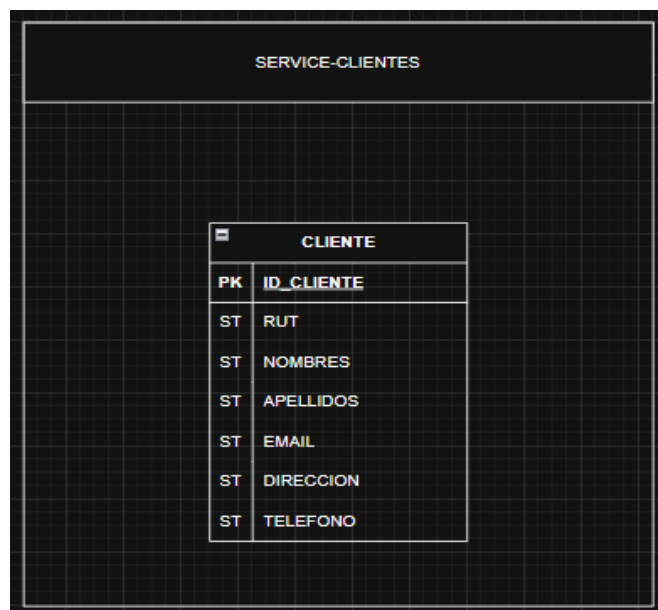
Especificación de Requisitos de Software (ERS)

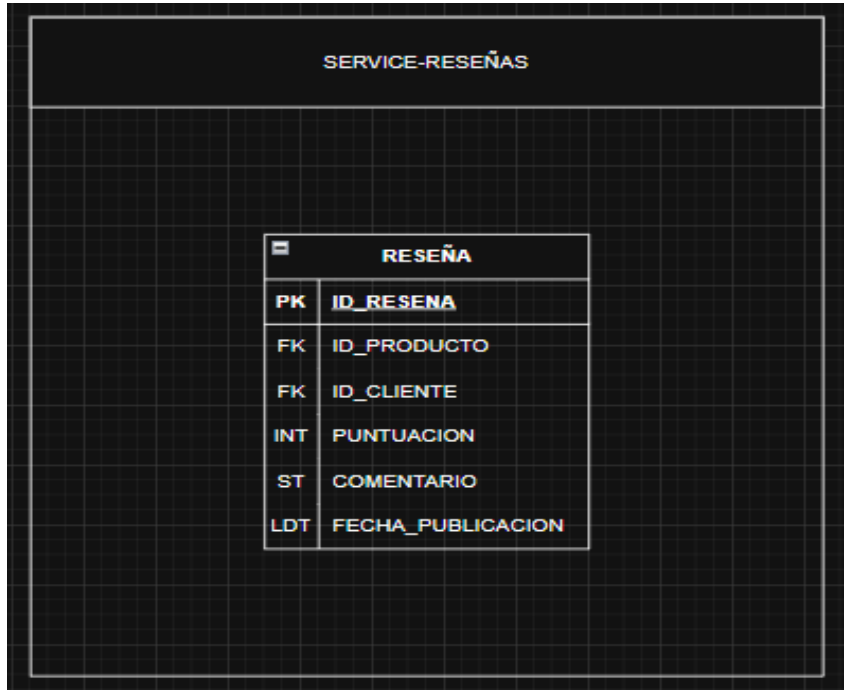
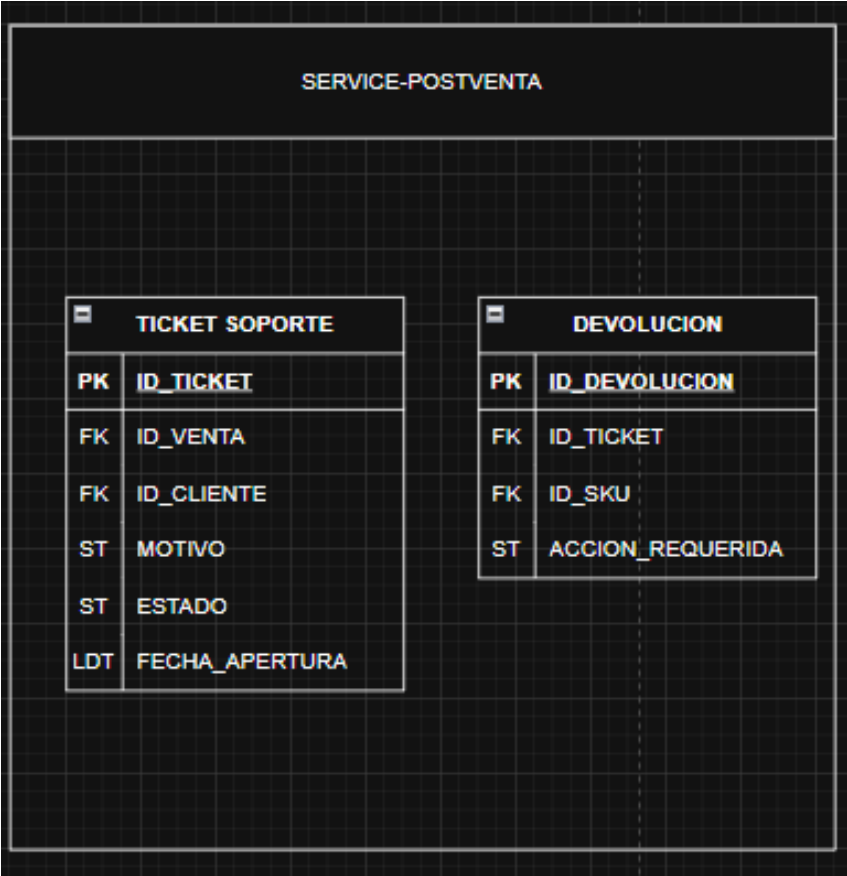
Arquitectura de 10 Microservicios + Autenticación

1. Modelo Entidad-Relación (MER) por Microservicio

La arquitectura se organiza en 10 microservicios de negocio, más un microservicio de autenticación centralizada. Cada microservicio mantiene su propia base de datos.







2. Planilla de Riesgos de la ERS

Esta matriz evalúa los riesgos técnicos y operativos durante el desarrollo e implementación del software, actualizada para considerar los riesgos propios de una arquitectura distribuida de microservicios.

ID	Descripción del Riesgo	Prob.	Impacto	Estrategia de Mitigación
R01	Inconsistencia de datos inicial: el inventario físico de bodega no coincide con la carga inicial del sistema en service-catalogo.	Alta	Alto	Realizar un inventario físico exhaustivo el día del paso a producción (conteo manual auditado).
R02	Mala definición de tallas/variantes: la tabla Variante_SKU no soporta tallas especiales (ej: Unitalla o calzado).	Medio	Alto	Diseñar Variante_SKU con valores alfanuméricos flexibles y campos nulos en atributos no aplicables.
R03	Resistencia al cambio: los vendedores prefieren el método antiguo y no registran ventas en tiempo real.	Alta	Medio	Crear una interfaz de punto de venta (POS) rápida e intuitiva. Capacitar al personal.
R04	Pérdida de datos: fallo en el servidor o base de datos que borra el historico de ventas o de cualquier microservicio.	Baja	Critico	Implementar copias de seguridad (backups) automáticas diarias por cada base de datos.
R05	Cuellos de botella en reportes: las consultas de estadísticas en service-ventas se vuelven lentas con miles de registros.	Medio	Medio	Crear índices en la base de datos para fechas y SKU. Optimizar consultas desde el inicio.
R06	Falla de comunicación entre microservicios: si un servicio cae (ej: service-catalogo), otros que dependen de él por WebClient (service-ventas, service-abastecimiento) fallan en cascada.	Medio	Alto	Implementar manejo de excepciones (try/catch) en cada llamada WebClient con mensajes de error controlados en vez de caídas del sistema.
R07	Token JWT expuesto o expirado durante operaciones críticas, impidiendo el acceso al sistema vía el api-gateway.	Medio	Alto	Definir tiempos de expiración razonables y manejar el código 401 de forma clara en el filtro del Gateway.
R08	Inconsistencia en cupones de promociones: aplicación de un cupón a una venta o cliente inexistente.	Baja	Medio	Validar mediante WebClient la existencia de la venta y el cliente antes de registrar la aplicación del cupón.
R09	Desincronización de stock entre abastecimiento y catalogo: una orden de compra marcada como recibida sin sumar correctamente el stock.	Baja	Alto	Usar @Transactional en las operaciones de suma/resta de stock para garantizar atomicidad.

3. Especificación de Requisitos de Software (ERS)

3.1. Propósito

Definir los requisitos funcionales y no funcionales para un sistema administrativo (ERP/Backoffice) de gestión de inventario, ventas, abastecimiento, promociones, logística, fidelización y postventa para DesireeStore, una tienda de ropa. El sistema se implementa bajo una arquitectura de microservicios independientes, cada uno con persistencia propia, comunicados entre sí mediante WebClient y expuestos a través de un API Gateway con autenticación centralizada mediante JWT.

3.2. Requisitos Funcionales (RF)

- **RF-01 Gestión de Catálogo:** el sistema (service-catálogo) debe permitir registrar, editar y eliminar productos, asignándoles múltiples variantes (tallas y colores) generando un SKU único para cada combinación.
- **RF-02 Control de Inventario:** el sistema (service-catálogo) debe permitir sumar, restablecer o descontar cantidades del inventario de cada SKU, además de identificar SKUs bajo su stock mínimo.
- **RF-03 Registro de Ventas:** el sistema (service-ventas) debe permitir registrar transacciones con uno o más detalles, descontando automáticamente el stock del SKU correspondiente en service-catalogo en tiempo real mediante WebClient.
- **RF-04 Alertas de Reposición:** el sistema (service-catalogo) debe identificar y exponer mediante un endpoint los SKUs que han alcanzado o bajado su nivel de stock mínimo configurado.
- **RF-05 Estadísticas de Ventas:** el sistema (service-ventas) debe generar un reporte de los SKUs más y menos vendidos en base al histórico de Detalle_Venta.
- **RF-06 Estadísticas de Existencias:** el sistema (service-catalogo) debe mostrar la distribución actual del stock, agrupado por producto y desglosado por talla y color.
- **RF-07 Gestión de Abastecimiento:** el sistema (service-abastecimiento) debe permitir registrar proveedores y ordenes de compra; al marcar una orden como recibida, debe sumar automáticamente el stock correspondiente en service-catalogo.
- **RF-08 Gestión de Promociones:** el sistema (service-promociones) debe permitir crear cupones de descuento con fecha de vigencia y límite de usos, validando su aplicación contra una venta y un cliente existentes antes de registrarla.
- **RF-09 Gestion de Clientes:** el sistema (service-clientes) debe permitir registrar, editar, eliminar y consultar clientes, validando que no exista más de un cliente con el mismo correo electrónico.
- **RF-10 Gestión de Envíos:** el sistema (service-envios) debe permitir registrar despachos asociados a una venta, consultando los datos de la venta correspondiente mediante WebClient.
- **RF-11 Gestión de Postventa:** el sistema (service-postventa) debe permitir registrar tickets de soporte y devoluciones asociadas a una venta y un cliente, sin alterar el registro original de la venta.
- **RF-12 Gestión de Fidelización:** el sistema (service-fidelización) debe permitir administrar una billetera de puntos por cliente, registrando movimientos de ganancia y canje.

- **RF-13 Gestión de Reseñas:** el sistema (service-resenas) debe permitir a los clientes registrar una calificación y comentario sobre un producto existente en el catálogo.
- **RF-14 Gestión de Combos:** el sistema (service-combos) debe permitir crear paquetes promocionales que agrupen distintos SKUs del catálogo bajo un precio especial, sin alterar el inventario individual de cada SKU hasta el momento de la venta.
- **RF-15 Autenticación y Autorización:** el sistema (service-auth) debe permitir el registro de usuarios administrativos con contraseña encriptada y la generación de un token JWT al iniciar sesión, el cual será validado por el api-gateway en cada petición a los microservicios protegidos.

3.3. Requisitos No Funcionales (RNF)

- **RNF-01 Usabilidad:** la interfaz de registro de ventas no debe requerir más de 4 clics para completarse, optimizando el tiempo de atención al cliente.
- **RNF-02 Rendimiento:** los informes de estadísticas de ventas de los últimos 30 días deben generarse y mostrarse en pantalla en menos de 3 segundos.
- **RNF-03 Seguridad:** el acceso al sistema debe estar protegido mediante autenticación JWT gestionada por service-auth y validada por el api-gateway; las contraseñas deben almacenarse encriptadas (BCrypt) en la base de datos.
- **RNF-04 Concurrencia:** el sistema debe soportar al menos 3 usuarios (ej: 2 vendedores y 1 administrador) operando y modificando el inventario simultáneamente sin generar bloqueos o inconsistencias.
- **RNF-05 Disponibilidad e Independencia:** cada microservicio debe poder ejecutarse, reiniciarse y fallar de forma independiente sin detener la operación completa del sistema, gracias al desacoplamiento por base de datos propia.
- **RNF-06 Interoperabilidad:** la comunicación entre microservicios debe realizarse exclusivamente mediante HTTP REST utilizando WebClient, manejando errores de conexión sin exponer fallas internas al usuario final.
- **RNF-07 Mantenibilidad:** cada microservicio debe seguir el patrón de arquitectura CSR (Controller-Service-Repository) con separación clara de responsabilidades por paquete.
- **RNF-08 Escalabilidad:** el api-gateway debe centralizar el punto de entrada único del sistema, permitiendo agregar nuevos microservicios sin modificar la lógica de los existentes.

4. Servicio de autenticación (service-auth)

4.1. ¿En qué consiste?

Su única responsabilidad es gestionar quién puede entrar al sistema y verificar que cada petición venga de alguien autorizado.

Tiene dos responsabilidades concretas:

1. Registrar usuarios → guarda un nombre de usuario, una contraseña encriptada y los roles que tiene esa persona (ADMIN, VENDEDOR, BODEGUERO).
2. Generar tokens → cuando alguien hace login correctamente, le entrega un "pase" digital (el token JWT) que demuestra que ya se identificó.

4.2. ¿Cómo se implementa la autenticación por token?

Se usa JWT (JSON Web Token). Es un token que viaja en cada petición y contiene la identidad del usuario firmada digitalmente, para que nadie pueda falsificarlo.

El flujo completo tiene 3 etapas:

Etapas 1: Registro

El **AuthService** toma la contraseña en texto plano y la encripta antes de guardarla, así en la base de datos nunca se ve la contraseña real, solo un hash ilegible.

Etapas 2: Login y generación del token

El **AuthService** busca al usuario, compara la contraseña si coincide, llama al **JwtService** para construir el token.

El token lleva 3 partes:

- Quién es (**setSubject**, el username)
- Qué puede hacer (**claim("roles", roles)**)
- Hasta cuándo es válido (**se crea un método**, en este caso 2 horas)

Todo esto se firma con una clave secreta (**jwt.secret**) usando un algoritmo, para que si alguien intenta editar el token, la firma deje de coincidir y se detecte la manipulación.

Etapa 3: Validación en cada petición

Aquí entra el **api-gateway**. Cada vez que llega una petición a cualquier microservicio protegido, el **AuthenticationFilter** la intercepta antes de dejarla pasar:

1. Revisa si viene el header **Authorization: Bearer <token>**
2. Si no viene → responde **401 Unauthorized** de inmediato, sin molestar a ningún microservicio
3. Si viene → decodifica el token con la misma clave secreta y verifica que la firma sea válida y no haya expirado
4. Si todo está bien → deja pasar la petición al microservicio correspondiente

5. Implementación por capas:

5.1. Comunicación entre microservicios: ¿cómo se implementa?

En nuestro proyecto de E-Commerce, la comunicación entre microservicios se implementó bajo un enfoque estrictamente síncrono basado en peticiones HTTP/REST, utilizando la herramienta reactiva Spring WebClient. Debido a que aplicamos el patrón de "Base de datos por servicio" para garantizar un bajo acoplamiento, ningún módulo tiene acceso directo a las tablas de otro. Por lo tanto, cuando un servicio necesita información externa, realiza una llamada de red directa y espera activamente la respuesta para continuar su flujo.

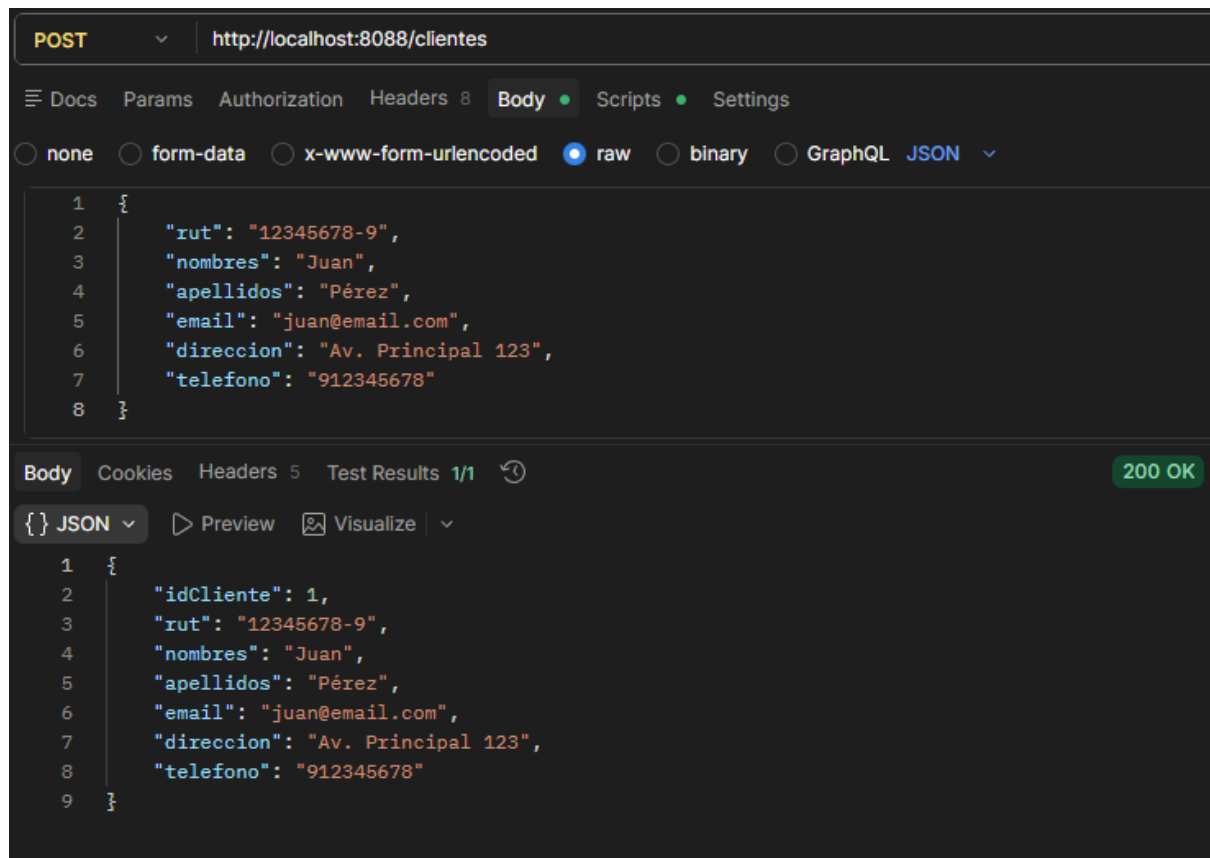
5.2. Evidencias:

SERVICE-CLIENTES

ClienteController (crear)

```
@PostMapping
public ResponseEntity<?> crear(@RequestBody Cliente cliente) {
    try {
        return ResponseEntity.ok(clienteService.guardar(cliente));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
```

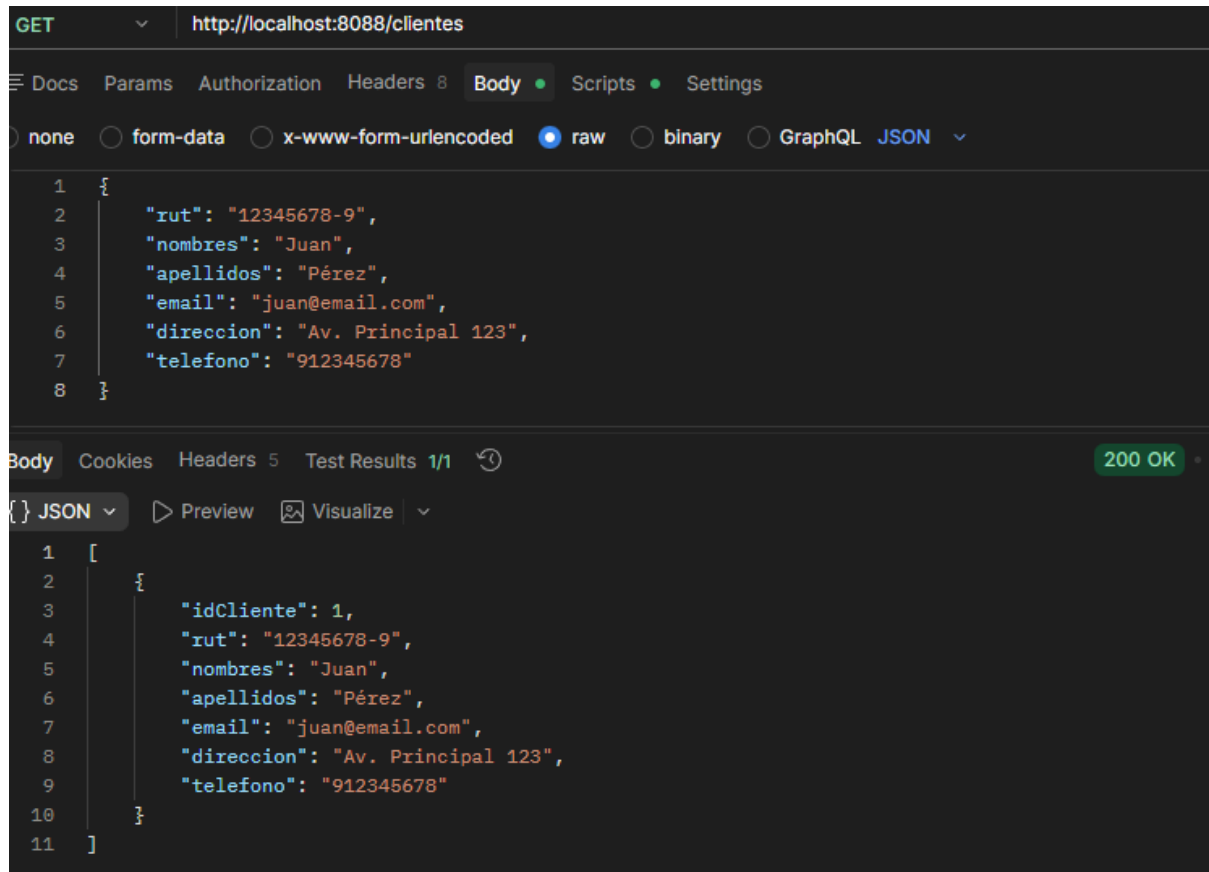
POSTMAN CREAR CLIENTE



ClienteController (Listar)

```
@GetMapping
public List<Cliente> listar() {
    return clienteService.listarTodos();
}
```

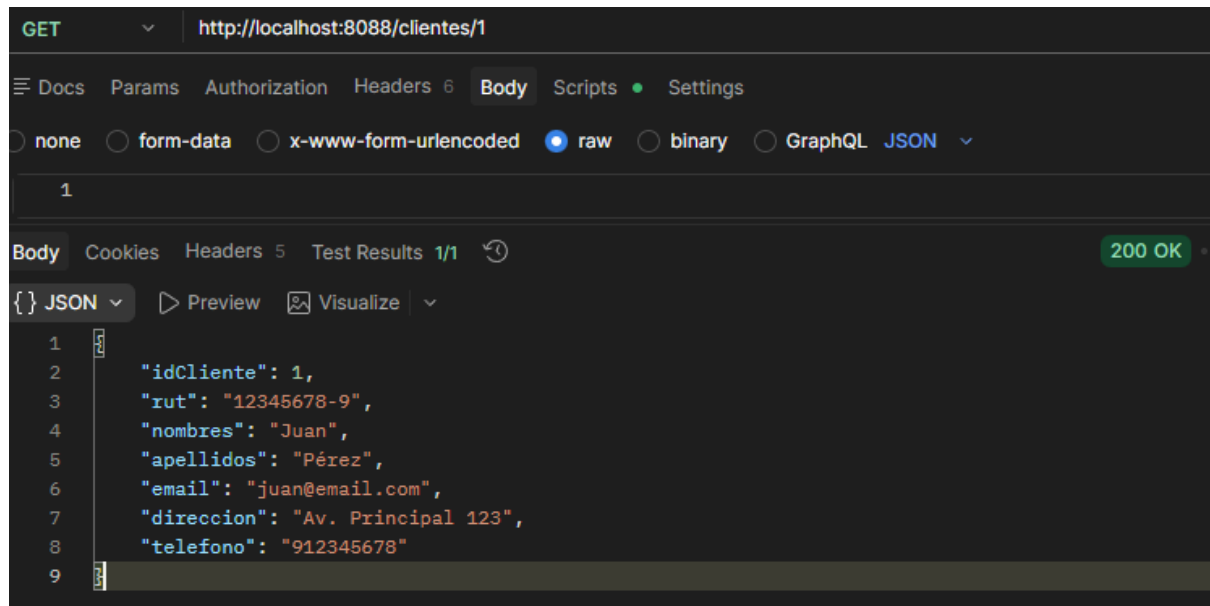
POST MAN LISTAR TODOS



Cliente Controller (obtener)

```
@GetMapping("/{id}")
public ResponseEntity<Cliente> obtener(@PathVariable Long id) {
    return clienteService.buscarPorId(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

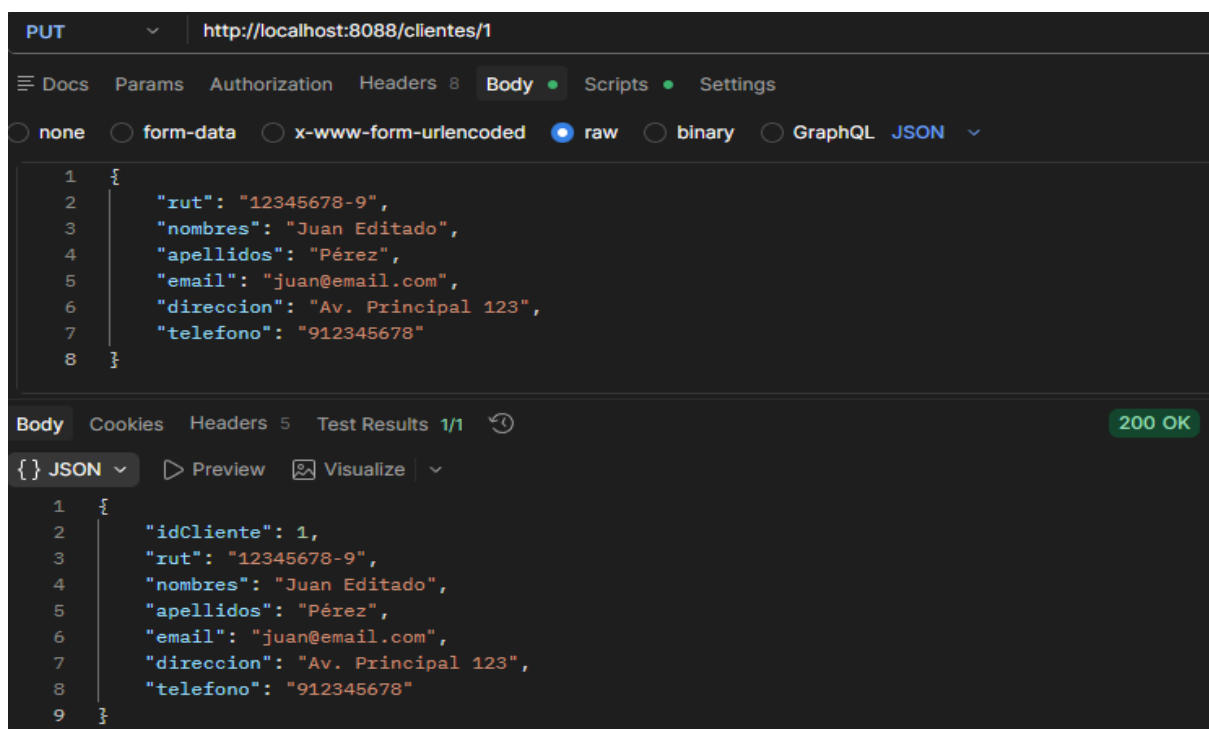
POSTMAN BUSCAR POR ID



ClienteController (Actualizar)

```
@PutMapping("/{id}")
public ResponseEntity<?> actualizar(@PathVariable Long id, @RequestBody Cliente cliente) {
    try {
        cliente.setIdCliente(id);
        return ResponseEntity.ok(clienteService.guardar(cliente));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
```

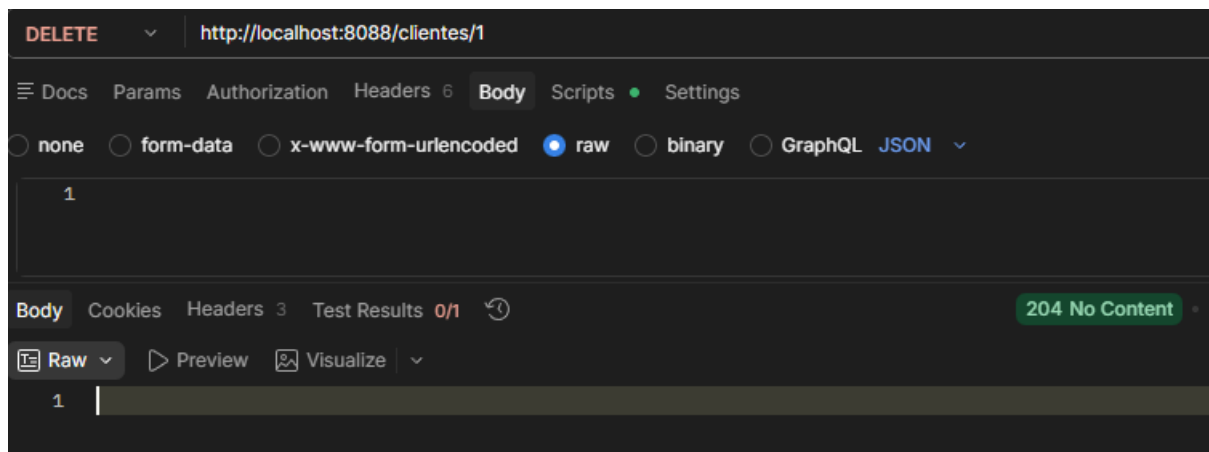
POSTMAN ACTUALIZAR



ClienteController (Eliminar)

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    clienteService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

POSTMAN ELIMINAR



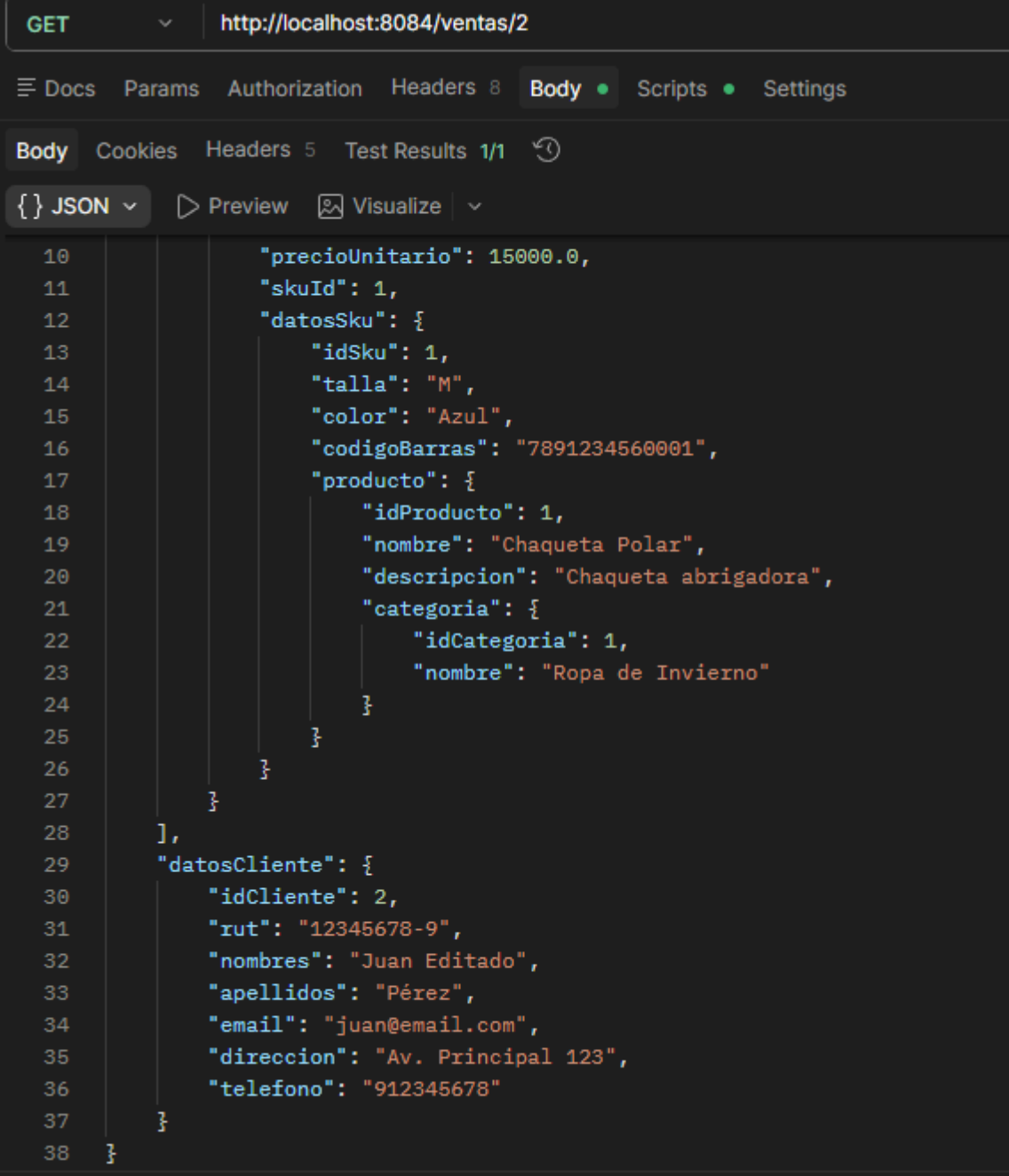
¿CÓMO SE IMPLEMENTA LA COMUNICACIÓN ENTRE MICROSERVICIOS?

Se implementa con WebClient, que es la herramienta reactiva de Spring que permite que un microservicio haga peticiones HTTP a otro, como si fuera Postman pero desde el código Java.

SERVICE-VENTAS COMUNICANDOSE CON SERVICE-CLIENTES

```
private Venta enriquecerConCliente(Venta venta) {
    if (venta.getClientId() != null) {
        try {
            Object cliente = webClientBuilder.build()
                .get()
                .uri("http://localhost:8088/clientes/" + venta.getClientId())
                .retrieve()
                .bodyToMono(Object.class)
                .block();
            venta.setDatosCliente(cliente);
        } catch (Exception e) {
            venta.setDatosCliente(datosCliente: "Cliente no disponible");
        }
    }
    return venta;
}
```

EJEMPLO EN POSTMAN CONSULTANDO LA VENTA Y ESTA TENIENDO LOS DATOS DEL CLIENTE



The screenshot shows the Postman interface with a GET request to `http://localhost:8084/ventas/2`. The response is in JSON format, displaying product details and client information. The JSON structure is as follows:

```
10     "precioUnitario": 15000.0,
11     "skuId": 1,
12     "datosSku": {
13         "idSku": 1,
14         "talla": "M",
15         "color": "Azul",
16         "codigoBarras": "7891234560001",
17         "producto": {
18             "idProducto": 1,
19             "nombre": "Chaqueta Polar",
20             "descripcion": "Chaqueta abrigadora",
21             "categoria": {
22                 "idCategoria": 1,
23                 "nombre": "Ropa de Invierno"
24             }
25         }
26     }
27 },
28 ],
29 "datosCliente": {
30     "idCliente": 2,
31     "rut": "12345678-9",
32     "nombres": "Juan Editado",
33     "apellidos": "Pérez",
34     "email": "juan@email.com",
35     "direccion": "Av. Principal 123",
36     "telefono": "912345678"
37 }
38 }
```

6. Swagger:

6.1. ¿Qué es? ¿En qué consiste? ¿Cómo se implementa en Spring Boot?

Swagger es una herramienta basada en la Especificación OpenAPI diseñada para documentar, visualizar y probar APIs RESTful de forma automatizada.

En nuestra arquitectura de Spring Boot, lo implementamos para generar un contrato vivo y accesible de nuestros microservicios.

Su funcionamiento se sostiene en tres pilares clave: primero, las Anotaciones Estructurales en el código fuente de Spring Boot (como `@Tag` y `@Operation` en controladores, y `@Schema` en modelos), que utilizamos para describir la lógica de cada ruta y validar las entidades.

Segundo, la Generación del Contrato, donde el sistema escanea estas etiquetas y crea dinámicamente un documento estandarizado en formato JSON que detalla todas las reglas, parámetros y respuestas de la API. Por último, la Interfaz Visual Interactiva (Swagger UI), que traduce ese código crudo en una página web amigable.

Esta interfaz es fundamental, ya que mediante su funcionalidad "Try it out", permite a desarrolladores y evaluadores ejecutar peticiones reales (POST, GET, PUT, DELETE) contra las bases de datos directamente desde el navegador, validando la robustez del código sin depender de herramientas externas.

6.2. Evidencias

DEPENDENCIA:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.7.0</version>
</dependency>
```

SERVICE-FIDELIZACION

MODELO BILLETERAPUNTOS:

```
@Entity
@Data
@Schema(description = "Billetera de puntos de un cliente")
public class BilleteraPuntos {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // FK Lógica -> Apunta al service-clientes
    @NotNull(message = "El ID del cliente es obligatorio")
    @Column(unique = true)
    @Schema(description = "ID del cliente", example = "5", requiredMode = Schema.RequiredMode.REQUIRED)
    private Long clienteId;

    @Schema(description = "Saldo actual", example = "1000")
    private Integer saldoActual = 0; // Inicializado en 0

    // Para enriquecer la respuesta más adelante con WebClient
    @Transient
    @Schema(description = "Datos del cliente traídos por WebClient", accessMode = Schema.AccessMode.READ_ONLY)
    private ClienteDTO datosCliente;
```

MODELO MOVIMIENTOPUNTOS:

```
@Entity
@Data
@Schema(description = "Registro de movimientos de la billetera")
public class MovimientoPuntos {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // FK Física -> Relación con la Billetera en esta misma BD
    @NotNull(message = "La billetera es obligatoria")
    @ManyToOne
    @JoinColumn(name = "billetera_id", nullable = false)
    private BilleteraPuntos billetera;

    @NotBlank(message = "El tipo de movimiento es obligatorio (ACUMULACION o CANJE)")
    @Schema(example = "ACUMULACION")
    private String tipoMovimiento;

    @NotNull(message = "La cantidad de puntos es obligatoria")
    @Schema(example = "500")
    private Integer cantidadPuntos;

    @NotBlank(message = "El motivo es obligatorio")
    @Schema(example = "Compra por Venta #5")
    private String motivo;

    private LocalDateTime fechaMovimiento;
}
```

MODELO CLIENTEDTO:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class ClienteDTO {
    private Long id;
    private String nombres;
    private String correo;
}
```

FIDELIZACION CONTROLLER:

```
@RestController
@RequestMapping("/api/fidelizacion")
@CrossOrigin(origins = "**")
@Tag(name = "Fidelización", description = "Gestión de puntos de clientes")
public class FidelizacionController {

    @Autowired
    private FidelizacionService service;

    @Operation(summary = "Crear Billetera", description = "Crea billetera con saldoActual 0")
    @PostMapping("/billetera")
    public ResponseEntity<BilleteraPuntos> crearBilletera(@Valid @RequestBody BilleteraPuntos billetera) {
        return ResponseEntity.ok(service.crearBilletera(billetera));
    }

    @Operation(summary = "Consultar Saldo por Cliente")
    @GetMapping("/billetera/cliente/{clienteId}")
    public ResponseEntity<BilleteraPuntos> obtenerBilletera(@PathVariable Long clienteId) {
        return service.obtenerPorClienteId(clienteId)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException("No existe billetera para este cliente"));
    }

    @Operation(summary = "Registrar Movimiento (Acumular/Canjear)")
    @PostMapping("/movimiento")
    public ResponseEntity<MovimientoPuntos> registrarMovimiento(@Valid @RequestBody MovimientoPuntos movimiento) {
        return ResponseEntity.ok(service.registrarMovimiento(movimiento));
    }

    @Operation(summary = "Obtener Billetera por ID", description = "Busca una billetera específica usando su ID de registro")
    @Operation(summary = "Actualizar saldo manualmente (PUT)", description = "Modifica directamente el saldo de puntos de una billetera")
    @PutMapping("/billetera/{id}")
    public ResponseEntity<BilleteraPuntos> actualizarSaldoManual(@PathVariable Long id, @RequestParam Integer nuevoSaldo) {
        return ResponseEntity.ok(service.actualizarSaldoManual(id, nuevoSaldo));
    }

    @Operation(summary = "Eliminar Billetera por ID", description = "Elimina por completo la billetera del sistema y confirma el éxito")
    @DeleteMapping("/billetera/{id}")
    public ResponseEntity<String> eliminarBilletera(@PathVariable Long id) {
        if (service.eliminarBilletera(id)) {
            return ResponseEntity.ok("Eliminado correctamente"); // Devuelve 200 OK con el mensaje en texto
        }
        return ResponseEntity.status(404).body("No se pudo eliminar: Billetera no encontrada");
    }
}
```

SWAGGER:

Fidelización Gestión de puntos de clientes		
POST	/api/fidelizacion/billetera	Crear Billetera
GET	/api/fidelizacion/billetera/{id}	Obtener Billetera por ID
PUT	/api/fidelizacion/billetera/{id}	Actualizar saldo manualmente (PUT)
DELETE	/api/fidelizacion/billetera/{id}	Eliminar Billetera por ID
GET	/api/fidelizacion/billetera/cliente/{clienteId}	Consultar Saldo por Cliente
POST	/api/fidelizacion/movimiento	Registrar Movimiento (Acumular/Canjear)

SERVICE-ENVIOS

MODELO DESPACHO:

```
@Entity
@Data
@Schema(description = "Modelo que representa un envío o despacho en el sistema de logística")
public class Despacho {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(description = "ID único autoincremental del despacho", example = "1")
    private Long id;

    // FK lógica -> Apunta al ID de la Venta en service-ventas
    @NotNull(message = "El ID de la venta es obligatorio")
    @Schema(description = "ID de la venta asociada (Referencia al Microservicio de Ventas)", example = "150", requiredMode = Schema.RequiredMode.REQUIRED)
    private Long ventaId;

    @NotBlank(message = "La dirección de destino no puede estar vacía")
    @Schema(description = "Dirección a la cual se enviará el paquete", example = "Av. Providencia 1234, Depto 502", requiredMode = Schema.RequiredMode.REQUIRED)
    private String direccionDestino;

    @NotBlank(message = "La comuna de destino es obligatoria")
    @Schema(description = "Comuna hacia donde se dirige el envío", example = "Providencia", requiredMode = Schema.RequiredMode.REQUIRED)
    private String comuna;

    @Schema(description = "Empresa externa encargada del transporte", example = "BlueExpress")
    private String empresaTransporte;

    @Column(unique = true)
    @Schema(description = "Código único de seguimiento alfanumérico", example = "TRK-550E8400")
    private String numeroSeguimiento;

    @Schema(description = "Estado actual del paquete en su ruta", example = "EN PREPARACION")
    private String estadoTracking; // Ej: "EN PREPARACION", "EN TRANSITO", "ENTREGADO"

    // para enriquecer los datos más adelante con WebClient
    @Transient
    @Schema(description = "Datos detallados de la venta traídos en tiempo real vía WebClient", accessMode = Schema.AccessMode.READ_ONLY)
    private VentaDTO datosVenta;
}
```

MODELO VENTADTO:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class VentaDTO {
    private Long id;
    private Long clienteId;
    private Double total;
    // Solo ponemos los campos que nos importan de la Venta original
}
```

DESPACHO CONTROLLER:

```
@RestController
@RequestMapping("/api/envios")
@CrossOrigin(origins = "**") // Permite que Swagger y el Gateway lo llamen
@Tag(name = "Despachos", description = "Operaciones relacionadas con la logística y envíos")
public class DespachoController {

    @Autowired
    private DespachoService service;

    @Operation(summary = "Registrar un nuevo despacho", description = "Crea un envío y genera un código de tracking automático")
    @PostMapping
    public ResponseEntity<Despacho> registrarDespacho(@Valid @RequestBody Despacho despacho) {
        return ResponseEntity.ok(service.registrarDespacho(despacho));
    }

    @Operation(summary = "Listar todos los despachos")
    @GetMapping
    public ResponseEntity<List<Despacho>> listarTodos() {
        return ResponseEntity.ok(service.listarTodos());
    }

    @Operation(summary = "Rastrear pedido", description = "Busca un envío por su código de seguimiento (Ej: TRK-XXX)")
    @GetMapping("/tracking/{numero}")
    public ResponseEntity<Despacho> rastrearPedido(@PathVariable String numero) {
        return service.rastrearPedido(numero)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException("No se encontró el envío con tracking: " + numero));
    }

    @Operation(summary = "Actualizar estado", description = "Modifica el estado del envío (Ej: EN CAMINO, ENTREGADO)")
    @PutMapping("/{id}/estado")
    public ResponseEntity<Despacho> actualizarEstado(@PathVariable Long id, @RequestParam String nuevoEstado) {
        return service.actualizarEstado(id, nuevoEstado)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException("No se encontró el envío con ID: " + id));
    }

    @Operation(summary = "Obtener despacho por ID", description = "Busca un despacho específico en la base de datos usando su ID numérico")
    @GetMapping("/{id}")
    public ResponseEntity<Despacho> obtenerPorId(@PathVariable Long id) {
        return service.obtenerPorId(id)
            .map(ResponseEntity::ok)
            .orElseThrow(() -> new RuntimeException("No se encontró el despacho con ID: " + id));
    }

    @Operation(summary = "Eliminar despacho por ID", description = "Elimina físicamente el registro del despacho de la base de datos y confirma el éxito")
    @DeleteMapping("/{id}")
    public ResponseEntity<String> eliminarDespacho(@PathVariable Long id) {
        if (service.eliminarDespacho(id)) {
            return ResponseEntity.ok("Eliminado correctamente"); // Devuelve 200 OK con el mensaje en texto
        }
        return ResponseEntity.status(404).body("No se pudo eliminar: Despacho no encontrado");
    }
}
```

SWAGGER:

Despachos Operaciones relacionadas con la logística y envíos

GET **/api/envios** Listar todos los despachos

POST **/api/envios** Registrar un nuevo despacho

GET **/api/envios/{id}** Obtener despacho por ID

DELETE **/api/envios/{id}** Eliminar despacho por ID

PUT **/api/envios/{id}/estado** Actualizar estado

GET **/api/envios/tracking/{numero}** Rastrear pedido

SERVICE-COMBOS

MODELO COMBO:

```
@Entity
@Data
@Schema(description = "Modelo que representa un paquete promocional o combo de productos")
public class Combo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(description = "ID único autoincremental del combo", example = "1")
    private Long id;

    @NotBlank(message = "El nombre del combo es obligatorio")
    @Schema(description = "Nombre comercial del combo", example = "Conjunto Buzo + Polerón", requiredMode = Schema.RequiredMode.REQUIRED)
    private String nombre;

    @NotBlank(message = "El tipo de combo es obligatorio")
    @Schema(description = "Categoría o tipo de promoción aplicada", example = "CONJUNTO", requiredMode = Schema.RequiredMode.REQUIRED)
    private String tipoCombo;

    @NotNull(message = "El precio fijo es obligatorio")
    @Schema(description = "Precio especial de venta por el paquete completo", example = "34990.0", requiredMode = Schema.RequiredMode.REQUIRED)
    private Double precioFijo;

    @Schema(description = "Estado de disponibilidad del combo en la tienda", example = "true")
    private Boolean estaActivo = true;

    @OneToMany(mappedBy = "combo", cascade = CascadeType.ALL, orphanRemoval = true)
    @Schema(description = "Lista detallada de los artículos que componen este combo promocional")
    private List<DetalleCombo> productosIncluidos;
}
```

MODELO DETALLECOMBO:

```
@Entity
@Data
@Schema(description = "Modelo que detalla los artículos específicos y cantidades dentro de un combo")
public class DetalleCombo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(description = "ID único autoincremental del detalle", example = "1")
    private Long id;

    @NotNull(message = "El SKU del producto es obligatorio")
    @Schema(description = "ID de la variante o producto (Referencia al Microservicio de Catálogo)", example = "1024", requiredMode = Schema.RequiredMode.REQUIRED)
    private Long skuId;

    @NotNull(message = "La cantidad es obligatoria")
    @Min(value = 1, message = "La cantidad mínima por producto debe ser 1")
    @Schema(description = "Unidades de este artículo que incluye el combo", example = "2", requiredMode = Schema.RequiredMode.REQUIRED)
    private Integer cantidad;

    @ManyToOne
    @JoinColumn(name = "combo_id", nullable = false)
    @JsonIgnore // Evita bucles infinitos en la serialización JSON
    @Schema(hidden = true) // Oculta este campo en Swagger para que no ensucie el JSON de entrada
    private Combo combo;

    @Transient
    @Schema(description = "Datos técnicos del SKU traídos en tiempo real vía WebClient", accessMode = Schema.AccessMode.READ_ONLY)
    private Object datosSku;
}
```

COMBO CONTROLLER:

```
@RestController
@RequestMapping("/api/combos")
@CrossOrigin(origins = "**")
@Tag(name = "Combos", description = "Gestión de paquetes promocionales")
public class ComboController {

    @Autowired
    private ComboService service;

    @Operation(summary = "Crear un nuevo combo promocional (POST)")
    @PostMapping
    public ResponseEntity<Combo> crearCombo(@RequestBody Combo combo) {
        return ResponseEntity.ok(service.crearCombo(combo));
    }

    @Operation(summary = "Obtener lista de combos activos")
    @GetMapping("/activos")
    public ResponseEntity<List<Combo>> obtenerCombosActivos() {
        return ResponseEntity.ok(service.listarCombosActivos());
    }

    @Operation(summary = "Obtener un combo por su ID (GET por ID)")
    @GetMapping("/{id}")
    public ResponseEntity<Combo> obtenerComboPorId(@PathVariable Long id) {
        return service.buscarPorId(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @Operation(summary = "Cambiar el estado activo/inactivo de un combo (PUT por ID)")
    @PutMapping("/{id}/estado")
    public ResponseEntity<?> cambiarEstado(@PathVariable Long id, @RequestParam Boolean estado) {
        try {
            return ResponseEntity.ok(service.cambiarEstadoCombo(id, estado));
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @Operation(summary = "Eliminar un combo de la base de datos (DELETE por ID)")
    @DeleteMapping("/{id}")
    public ResponseEntity<String> eliminarCombo(@PathVariable Long id) {
        if (service.eliminarCombo(id)) {
            return ResponseEntity.ok("Eliminado correctamente");
        }
        return ResponseEntity.status(404).body("No se pudo eliminar: Combo no encontrado");
    }
}
```

SWAGGER:

Combos Gestión de paquetes promocionales		
PUT	/api/combos/{id}/estado	Cambiar el estado activo/inactivo de un combo (PUT por ID)
POST	/api/combos	Crear un nuevo combo promocional (POST)
GET	/api/combos/{id}	Obtener un combo por su ID (GET por ID)
DELETE	/api/combos/{id}	Eliminar un combo de la base de datos (DELETE por ID)
GET	/api/combos/activos	Obtener lista de combos activos

7. Testing:

7.1. ¿En qué consiste? ¿Por qué es importante testear?

El Testing (o pruebas de software) consiste en el proceso de ejecutar y someter un sistema, o sus componentes individuales, a diferentes escenarios controlados para verificar que cumplen con los requisitos diseñados y no presentan defectos. En la práctica, implica estresar el código tanto con datos correctos (camino felices) como con datos inválidos para evaluar su comportamiento. En nuestro proyecto, esto lo aplicamos mediante pruebas unitarias puras utilizando JUnit y Mockito, lo que nos permitió aislar la capa de servicios (como calcular la fidelización o actualizar un ticket) y evaluar su lógica matemática y de negocio simulando el comportamiento de los repositorios sin tocar las bases de datos reales.

Su importancia es vital porque actúa como nuestro principal escudo de seguridad antes del despliegue. Previene que errores críticos o vulnerabilidades lleguen al usuario final, protegiendo la reputación y los ingresos del negocio. Además, al cumplir con el 60% de cobertura de código automatizada, garantizamos la mantenibilidad futura del sistema: si otro desarrollador modifica el proyecto más adelante y rompe alguna funcionalidad por accidente, las pruebas fallarán de inmediato en su consola, alertándolo del problema mucho antes de que afecte a la plataforma en vivo.

7.2. Explicar en qué consisten las pruebas unitarias (AAA)

Las pruebas unitarias consisten en aislar y evaluar la porción de código más pequeña de una aplicación (generalmente un método de la capa de Servicio) para verificar su lógica matemática o de negocio. Se ejecutan desconectadas de las bases de datos o servicios

externos mediante el uso de simulaciones (en nuestro caso, Mockito), garantizando que si una prueba falla, el error se encuentra estrictamente en la lógica de ese método y no en un fallo de conexión.

Para redactar estas pruebas de forma limpia, estandarizada y profesional, aplicamos el patrón estructural **AAA**, el cual divide cada test en tres fases secuenciales:

1. **Arrange (Preparar):** Es la fase de configuración del escenario. Aquí inicializamos los objetos, variables de prueba y programamos el comportamiento simulado de nuestras dependencias. Por ejemplo, le decimos a Mockito: *"cuando el servicio consulte la base de datos por el ticket ID 1, devuelve este ticket falso que acabo de crear"*.
2. **Act (Actuar):** Es la fase de ejecución real. Aquí llamamos e invocamos exclusivamente el método específico que queremos poner a prueba, pasándole los datos ficticios que preparamos en el paso anterior y capturando la respuesta en una variable.
3. **Assert (Verificar):** Es la fase de comprobación final. Utilizando las aserciones de JUnit (como `assertEquals` o `assertNotNull`), comparamos el resultado obtenido en la fase "Act" contra el resultado exacto que nosotros esperábamos obtener. Si coinciden, el código es confiable; si difieren, el test falla y hemos detectado un defecto antes de salir a producción.

7.3. Ejemplo con evidencia de capturas de pantalla de cada método implementado en cada capa de testeo explicando lo que se está testeando

SERVICE-VENTAS

```
@Mock
private VentaRepository ventaRepository;

@Mock
private DetalleVentaRepository detalleVentaRepository;

@Mock
private WebClient.Builder webClientBuilder;

@InjectMocks
private VentaService ventaService;
```

```

@Test
@DisplayName("Debería eliminar la venta llamando una vez al repositorio")
void eliminarVentaTest() {
    // --- PREPARACIÓN ---
    Long idVenta = 1L;

    // --- EJECUCIÓN ---
    ventaService.eliminar(idVenta);

    // --- VERIFICACIÓN ---
    verify(ventaRepository, times(1)).deleteById(idVenta);
}

```

```

@Test
@DisplayName("Debería retornar la lista completa de ventas registradas")
void listarTodasTest() {
    // --- PREPARACIÓN ---
    Venta venta1 = new Venta();
    venta1.setClienteId(clienteId: 1L);
    venta1.setTotal(total: 15000.0);

    Venta venta2 = new Venta();
    venta2.setClienteId(clienteId: 2L);
    venta2.setTotal(total: 25000.0);

    List<Venta> ventasMock = new ArrayList<>();
    ventasMock.add(venta1);
    ventasMock.add(venta2);

    when(ventaRepository.findAll()).thenReturn(ventasMock);

    // --- EJECUCIÓN ---
    List<Venta> resultado = ventaService.listarTodas();

    // --- VERIFICACIÓN ---
    assertEquals(2, resultado.size(), "Deben retornarse las 2 ventas registradas");
    verify(ventaRepository, times(1)).findAll();
}

```

```

@Test
@DisplayName("Debería fallar al guardar si la venta no tiene detalles")
void guardarVentaSinDetallesTest() {
    // --- PREPARACIÓN ---
    Venta ventaMock = new Venta();
    ventaMock.setClienteId(clienteId: 1L);
    ventaMock.setDetalles(detalles: null);

    when(ventaRepository.save(any(Venta.class))).thenReturn(ventaMock);

    // --- EJECUCIÓN ---
    Venta resultado = ventaService.guardar(ventaMock);

    // --- VERIFICACIÓN ---
    assertNull(resultado.getTotal(), "El total debe ser nulo o no calcularse si no hay detalles");
    verify(ventaRepository, times(1)).save(ventaMock);
}

```

Test Runner for Java

```

✓  🔍 VentaServiceTest $(symbol-namespace) com.dessirestore.service_ventas.s...
✓  📦 guardarVentaSinDetallesTest() $(symbol-class) VentaServiceTest < $(sym...
✓  📦 eliminarVentaTest() $(symbol-class) VentaServiceTest < $(symbol-namesp...
✓  📦 listarTodasTest() $(symbol-class) VentaServiceTest < $(symbol-namespace)..

```

SERVICE-CATALOGO

```

@Mock
private InventarioRepository inventarioRepository;

@InjectMocks
private CatalogoService catalogoService;

```



```

@Test
@DisplayName("Debería descontar stock correctamente cuando hay suficiente cantidad")
void descontarStockExitosoTest() {
    // --- PREPARACIÓN ---
    Long idSku = 1L;
    int cantidadADescontar = 5;

    VarianteSku sku = new VarianteSku();
    sku.setIdSku(idSku);

    Inventario inventarioMock = new Inventario();
    inventarioMock.setIdInventario(idInventario: 1L);
    inventarioMock.setVarianteSku(sku);
    inventarioMock.setCantidadActual(cantidadActual: 50);
    inventarioMock.setStockMinimo(stockMinimo: 10);

    when(inventarioRepository.findByVarianteSkuIdSku(idSku))
        .thenReturn(Optional.of(inventarioMock));
    when(inventarioRepository.save(any(Inventario.class)))
        .thenReturn(inventarioMock);

    // --- EJECUCIÓN ---
    Inventario resultado = catalogoService.descontarStock(idSku, cantidadADescontar);

    // --- VERIFICACIÓN ---
    assertEquals(45, resultado.getCantidadActual(), "El stock debe ser 45 después de descontar 5");
    verify(inventarioRepository, times(1)).save(inventarioMock);
}

```

```

@Test
@DisplayName("Debería lanzar excepción cuando el stock es insuficiente")
void descontarStockInsuficienteTest() {
    // --- PREPARACIÓN ---
    Long idSku = 1L;
    int cantidadADescontar = 100;

    VarianteSku sku = new VarianteSku();
    sku.setIdSku(idSku);

    Inventario inventarioMock = new Inventario();
    inventarioMock.setVarianteSku(sku);
    inventarioMock.setCantidadActual(cantidadActual: 10);
    inventarioMock.setStockMinimo(stockMinimo: 5);

    when(inventarioRepository.findByVarianteSkuIdSku(idSku))
        .thenReturn(Optional.of(inventarioMock));

    // --- VERIFICACIÓN ---
    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        catalogoService.descontarStock(idSku, cantidadADescontar);
    });

    assertEquals("Stock insuficiente para SKU: " + idSku, exception.getMessage());
}

```

```

@Test
@DisplayName("Debería retornar solo los SKUs bajo el stock mínimo en alertas")
void alertasReposicionTest() {
    // --- PREPARACIÓN ---
    Inventario inv1 = new Inventario();
    inv1.setCantidadActual(cantidadActual: 5);
    inv1.setStockMinimo(stockMinimo: 10); // 5 <= 10 → debe aparecer en alertas

    Inventario inv2 = new Inventario();
    inv2.setCantidadActual(cantidadActual: 50);
    inv2.setStockMinimo(stockMinimo: 10); // 50 > 10 → NO debe aparecer

    Inventario inv3 = new Inventario();
    inv3.setCantidadActual(cantidadActual: 3);
    inv3.setStockMinimo(stockMinimo: 5); // 3 <= 5 → debe aparecer en alertas

    when(inventarioRepository.findAll())
        .thenReturn(Arrays.asList(inv1, inv2, inv3));

    // --- EJECUCIÓN ---
    List<Inventario> alertas = catalogoService.alertasReposicion();

    // --- VERIFICACIÓN ---
    assertEquals(2, alertas.size(), "Solo deben aparecer 2 SKUs en alerta");
}

```

Test Runner for Java

- ✓  CatalogoServiceTest \$(symbol-namespace) com.dessirestore.service_cat
- ✓  alertasReposicionTest() \$(symbol-class) CatalogoServiceTest < \$(symbol-
- ✓  descontarStockExitosoTest() \$(symbol-class) CatalogoServiceTest < \$(sy
- ✓  descontarStockInsuficienteTest() \$(symbol-class) CatalogoServiceTest <

SERVICE-PROMOCIONES

```
@Mock
private CuponDescuentoRepository cuponRepository;

@Mock
private AplicacionCuponRepository aplicacionRepository;

@Mock
private WebClient.Builder webClientBuilder;

@InjectMocks
private PromocionesService promocionesService;
```

```
@Test
@DisplayName("Debería lanzar excepción cuando el cupón no está activo")
void aplicarCuponInactivoTest() {
    // --- PREPARACIÓN ---
    Long idCupon = 1L;

    CuponDescuento cuponMock = new CuponDescuento();
    cuponMock.setIdCupon(idCupon);
    cuponMock.setCodigoTexto(codigoTexto: "INVIERNO2026");
    cuponMock.setActivo(activo: false); // Cupón inactivo
    cuponMock.setFechaExpiracion(LocalDate.now().plusDays(30));
    cuponMock.setUsosMaximos(usosMaximos: 100);

    when(cuponRepository.findById(idCupon))
        .thenReturn(Optional.of(cuponMock));

    // --- VERIFICACIÓN ---
    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        promocionesService.aplicarCupon(idCupon, ventaId: 1L, clienteId: 1L);
    });

    assertEquals("El cupon no esta activo.", exception.getMessage());
}
```

```

@Test
@DisplayName("Debería lanzar excepción cuando el cupón ha expirado")
void aplicarCuponExpiradoTest() {
    // --- PREPARACIÓN ---
    Long idCupon = 1L;

    CuponDescuento cuponMock = new CuponDescuento();
    cuponMock.setIdCupon(idCupon);
    cuponMock.setActivo(activo: true);
    cuponMock.setFechaExpiracion(LocalDate.now().minusDays(1)); // Expirado ayer
    cuponMock.setUsosMaximos(usosMaximos: 100);

    when(cuponRepository.findById(idCupon))
        .thenReturn(Optional.of(cuponMock));

    // --- VERIFICACIÓN ---
    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        promocionesService.aplicarCupon(idCupon, ventaId: 1L, clienteId: 1L);
    });

    assertEquals("El cupon ha expirado.", exception.getMessage());
}

```

```

@Test
@DisplayName("Debería lanzar excepción cuando el cupón alcanzó el límite de usos")
void aplicarCuponUsosMaximosTest() {
    // --- PREPARACIÓN ---
    Long idCupon = 1L;

    CuponDescuento cuponMock = new CuponDescuento();
    cuponMock.setIdCupon(idCupon);
    cuponMock.setActivo(activo: true);
    cuponMock.setFechaExpiracion(LocalDate.now().plusDays(30));
    cuponMock.setUsosMaximos(usosMaximos: 2); // Máximo 2 usos

    // Simulamos que ya se usó 2 veces
    AplicacionCupon ap1 = new AplicacionCupon();
    AplicacionCupon ap2 = new AplicacionCupon();
    List<AplicacionCupon> aplicaciones = Arrays.asList(ap1, ap2);

    when(cuponRepository.findById(idCupon))
        .thenReturn(Optional.of(cuponMock));
    when(aplicacionRepository.findByCuponDescuentoIdCupon(idCupon))
        .thenReturn(aplicaciones);

    // --- VERIFICACIÓN ---
    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        promocionesService.aplicarCupon(idCupon, ventaId: 1L, clienteId: 1L);
    });

    assertEquals("El cupon ha alcanzado su limite de usos.", exception.getMessage());
}

```

```

@Test
@DisplayName("Debería retornar solo los cupones activos")
void listarCuponesActivosTest() {
    // --- PREPARACIÓN ---
    CuponDescuento coupon1 = new CuponDescuento();
    coupon1.setActivo(activo: true);

    CuponDescuento coupon2 = new CuponDescuento();
    coupon2.setActivo(activo: true);

    when(couponRepository.findByActivo(activo: true))
        .thenReturn(Arrays.asList(coupon1, coupon2));

    // --- EJECUCIÓN ---
    List<CuponDescuento> activos = promocionesService.listarCuponesActivos();

    // --- VERIFICACIÓN ---
    assertEquals(2, activos.size(), "Deben haber 2 cupones activos");
    verify(couponRepository, times(1)).findByActivo(activo: true);
}

```

Test Runner for Java

- ✓  PromocionesServiceTest \$(symbol-namespace) com.dessirestore.serv
- ✓  aplicarCuponExpiradoTest() \$(symbol-class) PromocionesServiceTest
- ✓  aplicarCuponInactivoTest() \$(symbol-class) PromocionesServiceTest
- ✓  aplicarCuponUsosMaximosTest() \$(symbol-class) PromocionesServ
- ✓  listarCuponesActivosTest() \$(symbol-class) PromocionesServiceTest

SERVICE-RESENAS

```
@Mock
private ResenaRepository repository;

@InjectMocks
private ResenaService service;
```

```
@Test
@DisplayName("Debería registrar la reseña asignándole la fecha de publicación actual")
void publicarResenaTest() {
    // 1. Arrange (Preparación)
    Resena resenaNueva = new Resena();
    resenaNueva.setProductoId(productoId: 10L);
    resenaNueva.setClienteId(clienteId: 45L);
    resenaNueva.setPuntuacion(puntuacion: 5);
    resenaNueva.setComentarioTexto(comentarioTexto: "Me encantó el producto, muy abrigador.");

    // Definimos el comportamiento del mock del repositorio
    when(repository.save(any(Resena.class))).thenAnswer(invocation -> {
        Resena r = invocation.getArgument(0);
        r.setId(id: 1L);
        return r;
    });

    // 2. Act (Ejecución)
    Resena resultado = service.publicarResena(resenaNueva);

    // 3. Assert (Verificación)
    assertNotNull(resultado, "La reseña guardada no debe ser nula");
    assertEquals(1L, resultado.getId(), "El ID retornado por la simulación debe ser 1");
    assertEquals(5, resultado.getPuntuacion(), "La puntuación debe coincidir");
    assertNotNull(resultado.getFechaPublicacion(), "El servicio debió asignarle automáticamente la fecha de publicación");

    // Verificar interacción
    verify(repository, times(1)).save(any(Resena.class));
}
```

Test Runner for Java

- ✓  ResenaServiceTest \$(symbol-namespace) com.dessirestore.serv
- ✓  publicarResenaTest() \$(symbol-class) ResenaServiceTest < \$(sym

SERVICE-FIDELIZACION

```
@Mock
private BilleteraPuntosRepository billeteraRepository;

@Mock
private MovimientoPuntosRepository movimientoRepository;

@InjectMocks
private FidelizacionService service;
```

```
@Test
@DisplayName("Debería sumar cantidadPuntos al saldoActual en una ACUMULACION")
void registrarMovimientoAcumulacionTest() {
    // 1. Arrange (Preparación)
    // Simulamos la Billetera existente en BD
    BilleteraPuntos billeteraMock = new BilleteraPuntos();
    billeteraMock.setId(id: 1L);
    billeteraMock.setClienteId(clienteId: 50L);
    billeteraMock.setSaldoActual(saldoActual: 1000); // Tiene 1000 puntos base

    // Simulamos el objeto que recibe el método (con la relación ManyToOne)
    BilleteraPuntos referenciaBilletera = new BilleteraPuntos();
    referenciaBilletera.setId(id: 1L);

    MovimientoPuntos movimientoRequest = new MovimientoPuntos();
    movimientoRequest.setBilletera(referenciaBilletera);
    movimientoRequest.setCantidadPuntos(cantidadPuntos: 500); // Sumará 500 puntos
    movimientoRequest.setTipoMovimiento(tipoMovimiento: "ACUMULACION");
    movimientoRequest.setMotivo(motivo: "Compra de prueba");

    // Reglas de Mockito
    when(billeteraRepository.findById(1L)).thenReturn(Optional.of(billeteraMock));
    when(movimientoRepository.save(any(MovimientoPuntos.class))).thenReturn(i -> i.getArgument(0));

    // 2. Act (Ejecución)
    MovimientoPuntos resultado = service.registrarMovimiento(movimientoRequest);

    // 3. Assert (Verificación)
    assertNotNull(resultado, "El movimiento no debe ser nulo");
    assertNotNull(resultado.getFechaMovimiento(), "Debe asignar fechaMovimiento");
    assertEquals(1500, billeteraMock.getSaldoActual(), "El saldoActual debió sumar 1000 + 500 = 1500");

    // Verificamos que se ejecutó el save() para actualizar la billetera
    verify(billeteraRepository, times(1)).save(billeteraMock);
}
```

Test Runner for Java

- ✓  FidelizacionServiceTest \$(symbol-namespace) c
- ✓  registrarMovimientoAcumulacionTest() \$(sym

SERVICE-ENVIOS

```
@Mock
private DespachoRepository repository;

@Mock
private WebClient.Builder webClientBuilder; // Mockeamos el WebClient

@InjectMocks
private DespachoService service;

@Test
@DisplayName("Debería rastrear un pedido y traer los datos de la Venta mediante WebClient")
void rastrearPedidoConWebClientTest() {
    // --- 1. PREPARACIÓN (Arrange) ---
    String numeroTracking = "TRK-12345";
    Long ventaId = 150L;

    // Simulamos el Despacho guardado en BD
    Despacho despachoMock = new Despacho();
    despachoMock.setId(id: 1L);
    despachoMock.setNumeroSeguimiento(numeroTracking);
    despachoMock.setVentaId(ventaId);

    // Simulamos la respuesta del otro microservicio (VentaDTO)
    VentaDTO ventaMock = new VentaDTO();
    ventaMock.setId(ventaId);
    ventaMock.setTotal(total: 50000.0);

    // Comportamiento del Repositorio
    when(repository.findByNumeroSeguimiento(numeroTracking)).thenReturn(Optional.of(despachoMock));

    // Comportamiento de la cadena de WebClient (Crucial para que no se quede pegado)
    WebClient webClient = Mockito.mock(WebClient.class);
    WebClient.RequestHeadersUriSpec uriSpec = Mockito.mock(WebClient.RequestHeadersUriSpec.class);
    WebClient.RequestHeadersSpec headersSpec = Mockito.mock(WebClient.RequestHeadersSpec.class);
    WebClient.ResponseSpec responseSpec = Mockito.mock(WebClient.ResponseSpec.class);
```

```
    when(webClientBuilder.build()).thenReturn(webClient);
    when(webClient.get()).thenReturn(uriSpec);
    when(uriSpec.uri(anyString())).thenReturn(headersSpec);
    when(headersSpec.retrieve()).thenReturn(responseSpec);

    // Entregamos el VentaDTO simulado
    when(responseSpec.bodyToMono(VentaDTO.class)).thenReturn(Mono.just(ventaMock));

    // --- 2. EJECUCIÓN (Act) ---
    // Aquí llamas al método de tu Service que busca por tracking y usa WebClient
    Optional<Despacho> resultadoOpt = service.rastrearPedido(numeroTracking);

    // --- 3. VERIFICACIÓN (Assert) ---
    assertTrue(resultadoOpt.isPresent(), "El despacho debe existir");
    Despacho resultado = resultadoOpt.get();

    assertNotNull(resultado.getDatosVenta(), "Los datos de la venta deben estar presentes (traídos por WebClient)");
    assertEquals(50000.0, resultado.getDatosVenta().getTotal(), "El total de la venta debe coincidir con el simulado");

    // Verificamos que se consultó a la BD
    verify(repository, times(1)).findByNumeroSeguimiento(numeroTracking);
}
```

Test Runner for Java

- ✓  DespachoServiceTest \$(symbol-namespac
- ✓  rastrearPedidoConWebClientTest() \$(syn

